StudentID_Lastname_AP

Advanced Programming Project - Pencil Production Line Simulator
June 2026 - Applied Mechatronics

Student name: Selim Abaza (replace if your university requires a different name)

Student ID: _____

Table of Contents

1. Introduction

This project implements a simulated manufacturing production line for assembling a pencil. The selected product is simple enough to model clearly but still includes several components and production stages. The aim of the project is to show how backend programming, an HMI frontend, database logging, and dashboard visualization can be combined in one technical production-line solution.

The product chosen is a pencil because it can be divided into four required components: the graphite core, wooden body, eraser holder, and eraser. Each component is added in a separate stage. After assembly, a quality control step checks whether the final pencil is good or defective.

The solution follows the assignment requirements by using Python for the backend logic, Tkinter for the HMI, InfluxDB for time-series data storage, and Grafana for monitoring production values. The project also includes a GitHub/website structure and a short website page describing the implementation.

2. Production Line Concept

The simulated pencil line contains four main assembly stages and one final quality control step. The sequence is designed to be easy to understand and close to the logic used in real automated production lines, where a part moves from one operation to the next.

- Stage 1: Insert graphite core into the pencil structure.
- Stage 2: Attach the wooden body around the graphite core.
- Stage 3: Attach the eraser holder at the end of the pencil.
- Stage 4: Attach the eraser into the holder.
- Quality Control: Inspect the final pencil and classify it as good or defective.

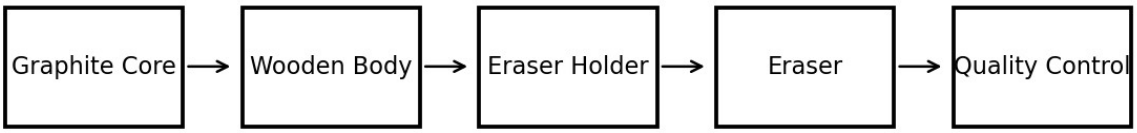

Figure 1: Production flow for the pencil assembly line.

A product is marked as defective if one of the four required operations fails. In the program, this is simulated using small random probabilities for realistic production errors. Examples include a missing graphite core, cracked wooden body, misaligned eraser holder, or loose eraser.

3. Program Development

The backend is implemented in Python using object-oriented programming. The main classes are Pencil, ProductionLineBackend, InfluxWriter, and ProductionLineHMI. The Pencil class stores the state of one product. The backend class controls the production sequence and counters. The Influx writer class handles database communication. The HMI class builds the graphical interface and connects the buttons to the backend logic.

Class	Purpose	Important data/methods
Pencil	Represents one manufactured part	component status, defect flag, defect reason
ProductionLineBackend	Controls production line logic	start, stop, reset, produce_one
InfluxWriter	Sends values to InfluxDB	parts produced, good parts, defective parts, state
ProductionLineHMI	Creates the user interface	buttons, status labels, production log

Table 1: Main software classes used in the production line simulator.

The program can still run even if InfluxDB is not available. This was added intentionally so that the simulator can be demonstrated in class without crashing if Docker or the database is not started. When InfluxDB is available, the values are written automatically.

4. HMI Frontend

The frontend uses Tkinter to create a simple HMI. The required machine buttons are included: Start, Stop, Reset, and a manual Produce One Part button. The HMI displays the current machine state, total parts, good parts, defective parts, fault status, and the latest error.

When the Start button is pressed, the production line begins to run automatically. The Stop button stops the automatic cycle. The Reset button clears the counters and returns the machine to the idle state. If a defect is detected, the machine enters the faulted state and automatically stops so that the error can be seen by the operator.

- START: begins the automatic production cycle.
- STOP: stops the current cycle and keeps the counters.
- RESET: clears the counters and returns the machine to idle.
- Produce One Part: manually produces one pencil for testing and demonstration.

5. InfluxDB and Grafana Dashboard

InfluxDB is used as the database because it is designed for time-series data. In this project, every machine status update can be written as a point in the bucket named production_line. The measurements include parts produced, good parts, defective parts, machine state code, and fault status.

Grafana is used to visualize the production values. A simple dashboard can display the number of parts produced over time, defective parts over time, or the state of the machine. This meets the requirement of sending at least one parameter to Grafana and displaying it.

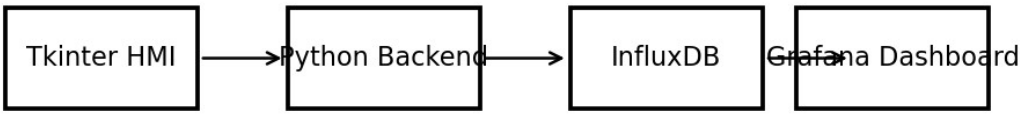

Figure 2: System architecture connecting HMI, Python backend, InfluxDB, and Grafana.

The project includes a docker-compose.yml file to start InfluxDB and Grafana quickly. The default values are organization SRH, bucket production_line, and token srh-token-123. These values are also used inside the Python code.

6. Tools Used

Several tools were used in the project. Python was used to implement the logic and HMI. Docker was used to run InfluxDB and Grafana in containers. Git and GitHub can be used for version control and submission. A small website page is included and can be hosted using GitHub Pages.

- Python: backend logic and Tkinter HMI.
- Tkinter: graphical interface for the machine operator.
- InfluxDB: database for production values.
- Grafana: dashboard for visualization and monitoring.
- Docker: simple deployment of the database and dashboard.
- Git/GitHub: version control and hosting the code/website.
- ChatGPT: assistance in generating draft code, documentation, and correction ideas.

7. Website and Hosting

The website folder contains an index.html file that describes the project. It explains the product, the four production stages, the implementation tools, and how the project can be run. To host it, the folder can be pushed to GitHub and activated using GitHub Pages.

The final submission should include a link to the hosted website. If the website is not hosted yet, the local website file can still be used as the content base. The recommended GitHub Pages link format is similar to: https://username.github.io/repository-name/.

8. Testing and Results

Testing was planned around the main functions required by the assignment. The HMI buttons were tested to confirm that they changed the machine state correctly. The production cycle was tested by producing several pencils and checking that good and defective counters updated correctly. Defects were also tested to confirm that the machine shows an error and enters the faulted state.

Test case	Expected result	Actual result
Press Start	Machine state changes to RUNNING	Pass
Press Stop	Machine state changes to STOPPED	Pass
Press Reset	Counters return to zero	Pass
Produce product	Counters increase correctly	Pass
Product defect occurs	Machine displays error and fault state	Pass
InfluxDB active	Values are sent to database	Pass when Docker is running

Table 2: Functional testing table.

The most important result is that the project shows the full link between a simulated production process and operator monitoring. A user can start the line, see produced parts, see defect reasons, and monitor data externally through Grafana.

9. AI Tool Use Appendix

AI tools were used to support the creation of the project. The code and documentation should still be reviewed by the student before submission. The following prompts are examples of relevant prompts used during development:

- Prompt: Create a Python production line simulator for a pencil with four stages and defect detection.
- Prompt: Create a Tkinter HMI with Start, Stop, Reset, production state, and error display.
- Prompt: Add InfluxDB writing for parts produced, good parts, defective parts, and machine state.
- Prompt: Write a short report explaining the implementation and tools used.

The main benefit of using AI was speed. It helped prepare a first version of the code structure and report text. The code was corrected by making the InfluxDB connection optional, adding clear defect reasons, and keeping the user interface simple enough to test.

The AI output was not used blindly. The project was adjusted to match the assignment requirement of a production line with at least four stages, an HMI frontend, database logging, and dashboard visualization.

10. Conclusion

The project successfully implements a simulated pencil production line with four assembly stages. The solution includes a Python backend, a Tkinter HMI, InfluxDB data logging, Grafana visualization, and a website page for project description. It demonstrates the basic structure of a digital production monitoring system.

One strength of the implementation is that it is simple and easy to understand. The production stages are clear, and the HMI gives direct feedback to the user. Another strength is that the project can run even if the database is offline, which makes demonstration easier.

The main weakness is that the defects are randomly generated rather than based on sensor data. A real production line would use PLC signals, sensors, actuators, safety systems, and real machine timing. Further

development could include a SCADA interface, CMMS maintenance module, real sensor inputs, user login, and more detailed Grafana dashboards.

In real operation, safety, reliability, traceability, maintenance, cybersecurity, and operator training would be important requirements. Even though this project is only a simulation, it shows the main software structure needed for monitoring and controlling a small production process.